

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 11

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

Goal **3**

1 Part 1: See the race **3**

2 Part 2: Fix it: mutex, then semaphore **6**

3 Part 3: A semaphore as a signal **11**

4 Part 4: A semaphore as a counter: the parking lot **13**

5 Optional: no lock at all **17**

Conclusion **19**

References **19**

Goal

Two threads that touch the same variable at the same time lose work. The purpose of this laboratory work is to see that loss happen, with a counter whose result is wrong and different at every run, and then to repair it twice: with a mutex and with a semaphore. The same semaphore is afterwards used in three different ways, which is the real subject of the work: as a lock, when it starts at 1 and lets one thread at a time into a critical section; as a signal, when it starts at 0 and makes one thread wait until another one announces that something is ready; and as a counter, when it starts at a larger number and limits how many threads may use a resource at once.

The work has four parts. The first part shows the race, with two threads that each add one to the same counter a million times and never obtain the expected two million. The second part repairs the counter with a mutex and then with a semaphore started at 1, and measures what the repair costs in time. The third part changes the problem from simultaneity to order: a reader thread must not read a message before a writer thread has written it, and a semaphore started at 0 enforces that. The fourth part models a parking lot with six cars and three places, where the semaphore counts the free places.

The programs were written and compiled in the directory os-lab11 of the course repository, with the command gcc -Wall -pthread. The option -O2 was deliberately not used, because the optimizer would keep the counter in a register instead of in memory and the race of the first part would disappear, which would hide exactly the phenomenon that had to be observed.

Part 1: See the race

Questions:

- counter++ looks like one step. Write the three steps the CPU really does.
- Describe, with those steps, how two threads can lose one addition.
- Why is the result different at every run?

Answers:

- The CPU reads the value of counter from the memory into a register, adds 1 to the register, and writes the register back into the memory. In assembly these are three separate instructions, a load, an add and a store, and between any two of them the thread can be interrupted, because the scheduler may take the processor away at any moment and because the two threads may in any case be running at the same time on two different cores.
- Suppose counter holds 100 and both threads execute the addition at the same time. The first thread loads 100 into its register. Before it writes anything back, the second thread also loads 100, adds 1 and stores 101. The first thread then continues with the value it had loaded, adds 1 to its own copy, which is still 100, and stores 101 as well. Two additions were performed, but the counter advanced by one: the

second store overwrote the first, and one addition was lost without any error being reported. The sequence is given in Table 1.

- Because the interleaving of the two threads is different at every run. Nothing in the program decides when a thread is interrupted or which core it runs on: that depends on the scheduler of the kernel, on the other processes of the system, on interrupts, and on the state of the caches of the processor at that moment. Every run therefore loses a different number of additions, and the result is a different wrong number each time. This is also what makes such a defect dangerous in practice: it does not fail in the same way twice, and it may well not fail at all on the machine of the programmer.

race.c – two threads are created with pthread_create, both executing the same function add, which adds one to the global variable counter a million times; the main thread waits for both with pthread_join and prints the result. The variable is global, so both threads see the same memory, and nothing in the program protects it. The source is given below.

```c
/* race.c - two threads add to the same counter, with no protection.
 * Build: gcc -Wall -pthread -o race race.c  (no -O2: keep counter++ in memory)
 * Run:   ./race   (several times)
 */
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;                   /* shared by both threads */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++;                  /* read, add 1, write back: three steps */
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

```
 GNU nano 8.7.1                          race.c *
/* race.c - two threads add to the same counter, with no protection.
* Build: gcc -Wall -pthread -o race race.c (no -O2: keep counter++ in memory)
* Run: ./race (several times)
*/
#include <stdio.h>
#include <pthread.h>
#define N 1000000
long counter = 0; /* shared by both threads */
void *add(void *arg) {
for (int i = 0; i < N; i++)
counter++; /* read, add 1, write back: three steps */
return NULL;
}
int main(void) {
pthread_t a, b;
pthread_create(&a, NULL, add, NULL);
pthread_create(&b, NULL, add, NULL);
pthread_join(a, NULL);
pthread_join(b, NULL);
printf("counter = %ld (expected %d)\n", counter, 2 * N);
```

Figure 1.1: The source of race.c in the editor

```
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab11$ gcc -Wall -pthread -o r
ace race.c
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab11$ ./race
counter = 1000000 (expected 2000000)
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab11$ ./race
counter = 1559265 (expected 2000000)
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab11$ ./race
counter = 1293920 (expected 2000000)
```

Figure 1.2: Three runs of race, three different wrong results

gcc -Wall -pthread -o race race.c ; ./race – the program was compiled without any warning and run three times, with the results 1000000, 1559265 and 1293920, against the expected 2000000. None of the three is correct, and no two of them are equal, which is the practical proof of the third answer. The first run is the most spectacular of the three: the counter advanced by exactly one million, as if one of the two threads had done nothing at all, which happens when the two threads run almost in lockstep and each store of one thread is immediately overwritten by the store of the other. The losses of the other two runs, about 440000 and about 706000 additions, show that the amount of lost work is itself random.

Step	Thread A	Thread B	counter
1	loads 100 into its register		100
2		loads 100 into its register	100
3		adds 1: its register holds 101	100
4		stores 101	101
5	adds 1: its register holds 101		101
6	stores 101		101

Table 1: Two additions, one increment: how an addition is lost

Table 1 shows that nothing illegal happened: every instruction did exactly what it was told. The defect is that the three steps of one addition were allowed to be interleaved with the three steps of another, and the only repair is to forbid that interleaving.

Part 2: Fix it: mutex, then semaphore

Questions:

- Run mutex and sem three times each. Is the answer always 2 000 000?
- What would happen in sem.c with sem_init(&s, 0, 2)? Try it and explain.
- Why is the locked version slower?

Answers:

- Yes. Every run of both programs printed counter = 2000000 (expected 2000000), and this is not luck: the result is now guaranteed rather than probable. Between the lock and the unlock only one thread can be inside, so the three steps of an addition can no longer be interleaved with the three steps of another, and no store can overwrite a value that another thread has not yet used. The result of the first part was different at every run precisely because it depended on the interleaving; here the interleaving no longer matters, so the output stops being random.
- The counter would be wrong again, exactly as in race.c. A semaphore started at 2 allows two threads to pass sem_wait at the same time, and the program has exactly two threads, so neither of them would ever wait: the semaphore would never reach 0, both would be inside the critical section together and the additions would be lost again. The start value of a semaphore is the number of threads allowed in at once, so for mutual exclusion it must be 1 and nothing else.

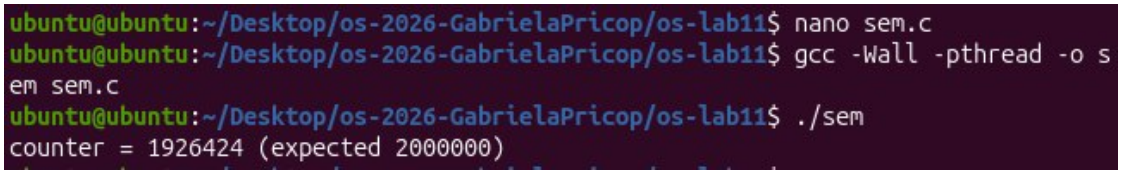


Figure 2.1: What happens with sem_init(&s, 0, 2)

- Because the protection has a price that is paid two million times. Each addition, which is three instructions, is now surrounded by a lock and an unlock, which are more instructions than the addition itself, and when the two threads collide the loser does not merely wait: it is put to sleep by the kernel and woken up again later, which is a system call and a context switch. Moreover, the variable and the lock travel between the caches of the two cores, so every passage of the lock from one thread to the other costs a transfer of a cache line. The measured times confirm this: the unprotected and wrong program took 0.013 s, the mutex 0.104 s and the semaphore 0.259 s, as collected in Table 2.

mutex.c – the same program as before, with the shared variable now surrounded by a mutex created statically with PTHREAD_MUTEX_INITIALIZER. The line between pthread_mutex_lock and pthread_mutex_unlock is the critical section: a thread that finds the lock taken stops at the lock call and continues only when the other thread has released it. The source is given below.

```c
/* mutex.c - the same counter, now protected by a mutex.
 * Build: gcc -Wall -pthread -o mutex mutex.c
 */
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);      /* only one thread past this line */
        counter++;                      /* the critical section */
        pthread_mutex_unlock(&lock);    /* let the other one in */
    }
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```



Figure 2.2: The beginning of mutex.c in the editor

nano mutex.c – only three lines differ from race.c: the declaration of the lock, the call that takes it and the call that releases it. The variable counter and the loop are untouched, which shows that the defect of the first part was not in the arithmetic but in the absence of a rule about who may touch the memory and when.

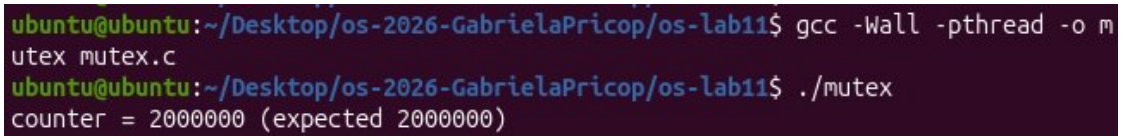


Figure 2.3: The mutex version always prints the expected two million

gcc -Wall -pthread -o mutex mutex.c ; ./mutex – the output is counter = 2000000 (expected 2000000). Not a single addition is lost, and the number no longer changes from one run to the next.

sem.c – the same repair, made with a semaphore instead of a mutex. A semaphore is a number that never goes below zero, with two operations: sem_wait takes one unit and blocks if there is none, sem_post gives one back. Started at 1, it behaves exactly like a lock: the first thread takes the unit and the value becomes 0, so the second one waits at sem_wait until the first calls sem_post. The second argument of sem_init, which is 0 here, means that the semaphore is shared between the threads of one process and not between processes. The source is given below.

```c
/* sem.c - the same counter, protected by a semaphore that
   starts at 1.
 * Build: gcc -Wall -pthread -o sem sem.c
 */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 1000000

long counter = 0;
sem_t s;

void *add(void *arg) {
```

```
    for (int i = 0; i < N; i++) {
        sem_wait(&s);      /* 1 -> 0: I go in; at 0: wait */
        counter++;
        sem_post(&s);      /* 0 -> 1: the next one may go in */
    }
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 1);    /* 0 = shared between threads, 1 =
   start value */
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    sem_destroy(&s);
    return 0;
}
```

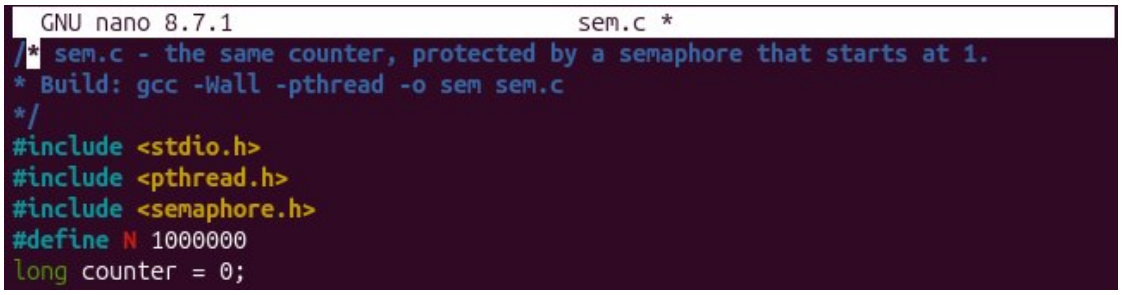


Figure 2.4: The beginning of sem.c in the editor

nano sem.c – the header semaphore.h is added and the mutex is replaced by a variable of type sem_t, which must be initialized at run time with sem_init and released at the end with sem_destroy, unlike the mutex, which could be initialized statically.

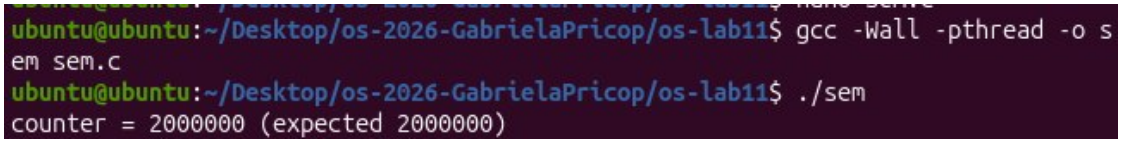


Figure 2.5: The semaphore version gives the same correct result

gcc -Wall -pthread -o sem sem.c ; ./sem – the output is again counter = 2000000 (expected 2000000). The two repairs are therefore equivalent for this problem, which is the point of the exercise: a semaphore started at 1 is one of the ways to build a lock.

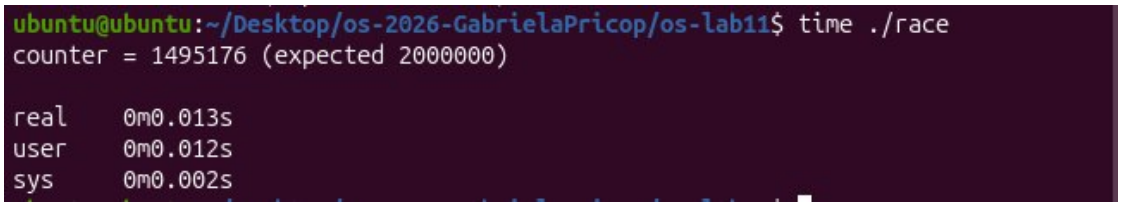


Figure 2.6: The time of the unprotected version, which is fast and wrong

time ./race – the unprotected program finished in 0.013 s of real time, with 0.012 s in the user mode and 0.002 s in the kernel, and printed 1495176, one more wrong result. It is the fastest of the three precisely because it does nothing to protect the counter.

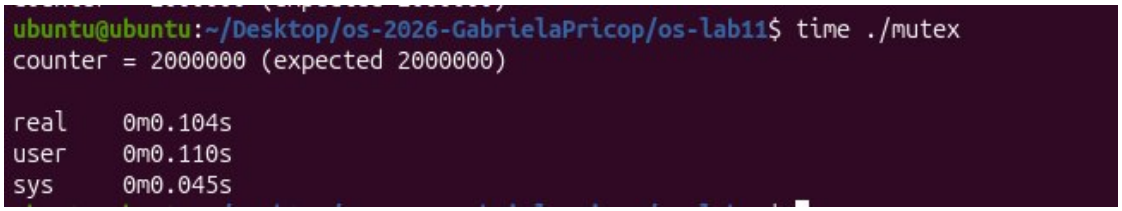


Figure 2.7: The time of the version protected by the mutex

time ./mutex – the correct version took 0.104 s of real time, that is eight times longer, with 0.110 s of user time and 0.045 s of system time. The user time is larger than the real time because the two threads ran at the same time on two cores, and the time of each of them is counted separately.

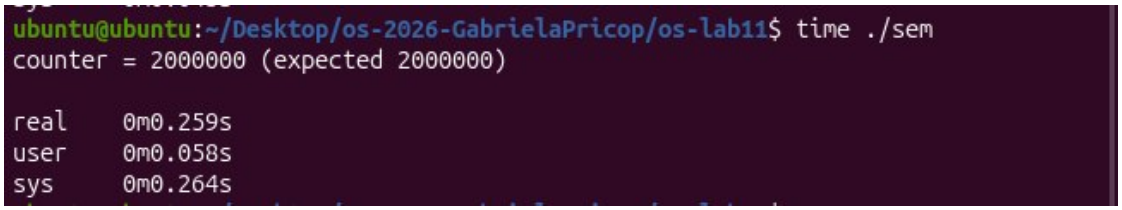


Figure 2.8: The time of the version protected by the semaphore

time ./sem – the semaphore took 0.259 s, which is about two and a half times the mutex. The distribution of that time is the interesting part: 0.058 s in the user mode but 0.264 s in the kernel, while the mutex spent only 0.045 s there. The reason is that a mutex of the pthread library tries first to take the lock entirely in the user space, with one atomic instruction, and calls the kernel only when it really has to wait, whereas the semaphore goes through the kernel much more often. The three measurements are collected in Table 2.

Program	**Result**	**real**	**user**	**sys**	**Correct**
./race	1495176	0.013 s	0.012 s	0.002 s	no
./mutex	2000000	0.104 s	0.110 s	0.045 s	yes
./sem	2000000	0.259 s	0.058 s	0.264 s	yes

Table 2: The price of the protection, measured with time

Table 2 shows the trade that every synchronization makes: the fast program is the one that is wrong, and the correct ones pay for their correctness in time. It also shows that the two correct solutions are not equally expensive, and that the difference is almost entirely in the system time, that is, in how often each of them has to ask the kernel for help.

Part 3: A semaphore as a signal

Questions:

- Change the start value to 1 (sem_init(&ready, 0, 1)), compile, run. What does B read now, and why?
- Put it back to 0. In one sentence: what does a semaphore that starts at 0 mean?

Answers:

- B reads an empty message. With the start value 1 the door is already open: sem_wait finds the value 1, lowers it to 0 and returns immediately, so B does not wait at all and prints its line while A is still sleeping inside its one second of pretended work. The array message is a global variable, which the C language fills with zeros before the program starts, so B reads an empty string and prints B: I read "". A writes hello afterwards, into a message that nobody is waiting for any more, and its line appears after the one of B. Nothing crashes and no error is reported, which is what makes this kind of defect hard to notice: the only visible sign is that the output is in the wrong order and the message is empty.

```
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab11$ ./order
B: waiting for the message...
B: I read ""
A: message written
```

Figure 3.1: Enter Caption

- A semaphore that starts at 0 means that nothing has happened yet, so whoever waits on it is blocked until another thread announces, with sem_post, that the thing has been done.

order.c – the problem of this part is no longer that two threads touch the same variable at the same time, but the order in which they act: the reader must not read before the writer has written. The semaphore ready is initialized at 0, the reader is deliberately created first, and it blocks at sem_wait; the writer sleeps one second, writes the message, prints its line and calls sem_post, which raises the value from 0 to 1 and wakes the reader. The source is given below.

```c
/* order.c - thread B must wait until thread A has written the
   message.
 * A semaphore that starts at 0 is a signal: sem_wait blocks
   until someone posts.
 * Build: gcc -Wall -pthread -o order order.c
 */
#include <stdio.h>
```

```c
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

char message[32];
sem_t ready;                    /* starts at 0: nothing is ready
    yet */

void *writer(void *arg) {
    sleep(1);                   /* pretend the work takes a while
    */
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready);           /* 0 -> 1: wake up B */
    return NULL;
}

void *reader(void *arg) {
    printf("B: waiting for the message...\n");
    sem_wait(&ready);           /* blocks while the value is 0 */
    printf("B: I read \"%s\"\n", message);
    return NULL;
}

int main(void) {
    sem_init(&ready, 0, 0);
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);   /* B starts first,
    on purpose */
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    sem_destroy(&ready);
    return 0;
}
```

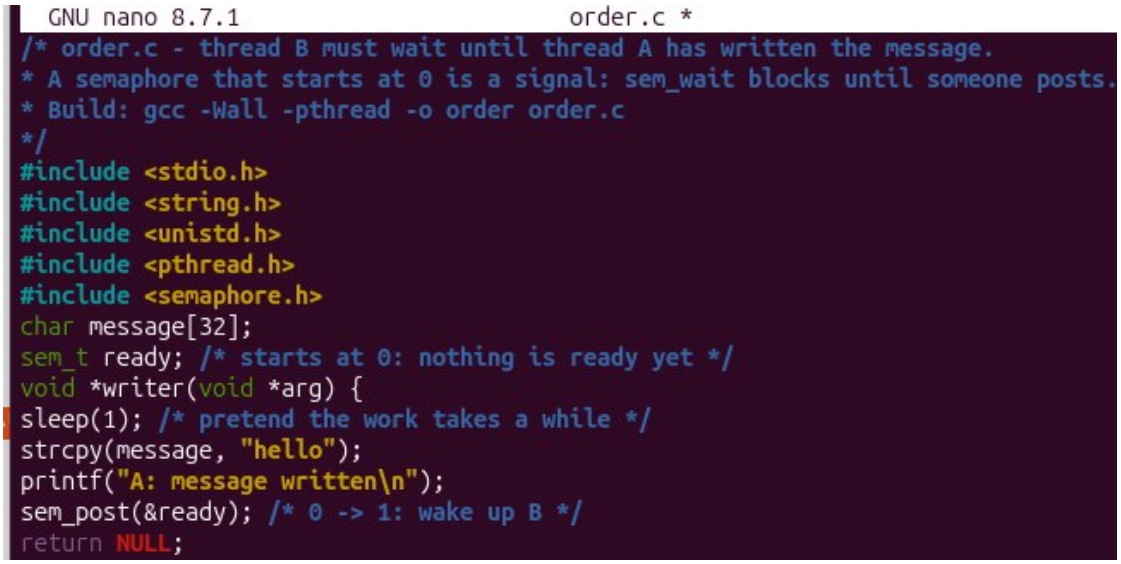


Figure 3.2: The beginning of order.c in the editor

nano order.c – the comment at the top states the idea in one line: a semaphore that starts at 0 is a signal. The call sleep(1) in the writer is not part of the mechanism; it only makes the delay long enough to be seen by a human, and it also guarantees that the reader really does arrive first, so that the experiment tests the semaphore and not the speed of the two threads.

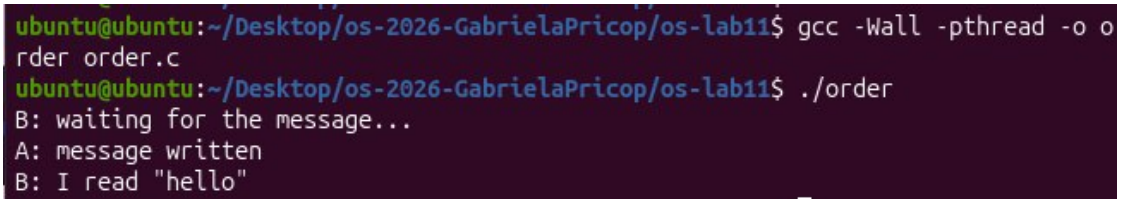


Figure 3.3: The three lines appear in the right order, with a pause in the middle

gcc -Wall -pthread -o order order.c ; ./order – the output is exactly the expected one: B: waiting for the message... appears at once, then nothing happens for one second, then A: message written and immediately after it B: I read "hello". The pause is visible to the eye and it is the proof that the reader was really blocked: it was not busy checking the variable in a loop, it was sleeping, and the kernel woke it up only when sem_post was called. A loop that keeps testing a variable would give the same order but would burn a whole core for one second, which is the difference between waiting and spinning.

Part 4: A semaphore as a counter: the parking lot

Questions:

- What is the largest value of "inside" you see? Which line of code guarantees it?
- Set PLACES to 1. What does the semaphore become?

Answers:

- The largest value is 3, and it is reached several times during the run. It is guaranteed by the pair formed by sem_init(&places, 0, PLACES), which starts the semaphore at 3, and sem_wait(&places), which every car must pass before entering: the semaphore never goes below zero, so at most three cars can hold a unit at the same time and the fourth one blocks until somebody calls sem_post. The mutex around the printing does not limit anything; it only prevents the increment of the variable inside and its printf from being separated, so that the printed numbers are consistent. Without the semaphore the mutex alone would let all six cars in, one at a time but all of them inside.

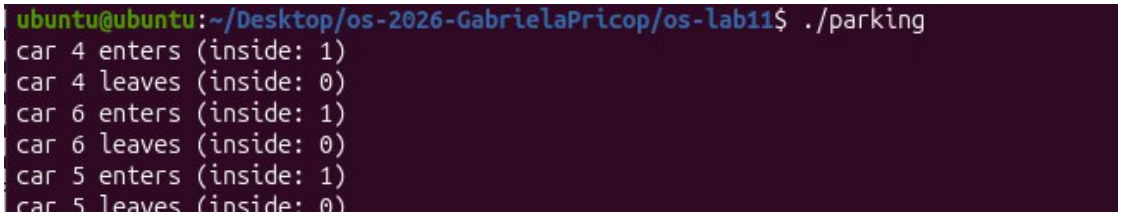


Figure 4.1: What happens when places is 1

- It becomes a mutex, that is, an ordinary lock. With a single place the semaphore alternates between 1 and 0 and allows exactly one car inside at a time, which is the situation of the second part: the parking lot becomes a critical section, the cars enter strictly one after another and the value of inside never exceeds 1.

parking.c – six threads, one for each car, share a semaphore initialized at three, the number of places. A car takes a place with sem_wait, enters, sleeps one second as if it were parked, leaves, and gives the place back with sem_post. The identifier of the car is passed to the thread as the argument of pthread_create, converted to a pointer and back, which is why the type is long. The source is given below.

```c
/* parking.c - 6 cars, 3 places. A semaphore that starts at 3
   counts the free places.
 * Build: gcc -Wall -pthread -o parking parking.c
 */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define PLACES 3
#define CARS   6

sem_t places;                   /* free places */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int inside = 0;                 /* only for printing */

void *car(void *arg) {
    long id = (long)arg;
    sem_wait(&places);          /* take a place, or wait */
    pthread_mutex_lock(&lock);
```

```c
    inside++;
    printf("car %ld enters (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);

    sleep(1);                   /* parked */

    pthread_mutex_lock(&lock);
    inside--;
    printf("car %ld leaves (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sem_post(&places);          /* give the place back */
    return NULL;
}

int main(void) {
    sem_init(&places, 0, PLACES);
    pthread_t t[CARS];
    for (long i = 0; i < CARS; i++)
        pthread_create(&t[i], NULL, car, (void *)(i + 1));
    for (int i = 0; i < CARS; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&places);
    return 0;
}
```



Figure 4.2: The beginning of parking.c in the editor

nano parking.c – the two mechanisms of the program have two different roles, and the comments say so: the semaphore counts the free places and decides who may enter, while the mutex exists only so that the variable inside and the line that prints it stay together. They are not interchangeable here: a mutex cannot count to three, and a semaphore used alone would still allow two cars to interleave their increment and their printf, which would print numbers that never existed.

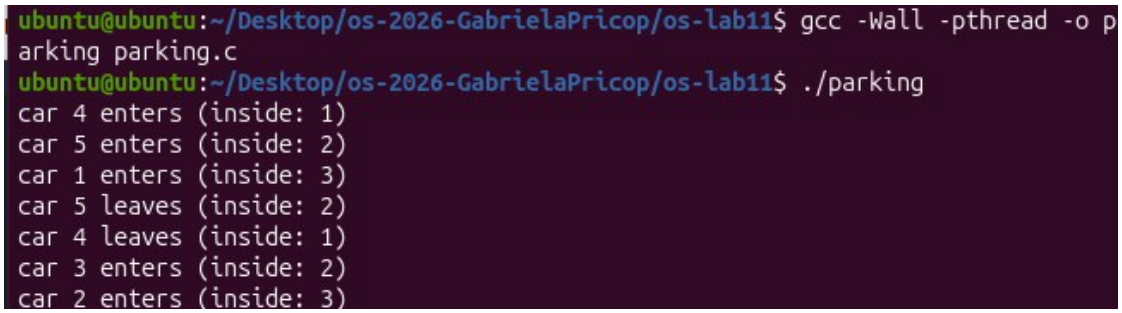


Figure 4.3: The parking lot never holds more than three cars

gcc -Wall -pthread -o parking parking.c ; ./parking – the order of the cars is not the order of their numbers, because the threads are started almost at the same time and the one that reaches sem_wait first gets the place: the first three to enter are the cars 4, 5 and 1, which fill the lot and bring inside to 3. The cars 5 and 4 then leave, which frees two places, and the cars 3 and 2 enter immediately and bring the count back to 3. The value of inside rises to 3, falls, and rises to 3 again, but never reaches 4, which is the whole point of the experiment. The beginning of the run is summarized in Table 3.

Line	**Event**	**inside**	**State of the semaphore**
1	car 4 enters	1	2 free places left
2	car 5 enters	2	1 free place left
3	car 1 enters	3	0: the lot is full, the others wait
4	car 5 leaves	2	1 free place again
5	car 4 leaves	1	2 free places
6	car 3 enters	2	1 free place
7	car 2 enters	3	0: full again

Table 3: The first lines of the run: the count never passes three

Table 3 shows the semaphore working as a counter rather than as a lock. The same object has now been used in the three ways of this laboratory work, which are collected in Table 4.

Start value	**Role**	**What it guarantees**	**Where it was used**
0	signal	a thread waits until another one announces an event	order.c, the reader waits for the message
1	lock	only one thread inside the critical section	sem.c, around counter++
n	counter	at most n threads use the resource at once	parking.c, with n = 3 places

Table 4: The same semaphore, read in three ways

Table 4 shows that the three uses are not three different tools but one rule applied to

three different numbers: the value of the semaphore is how many threads may still pass, and the only thing that changes is where that number starts.

Optional: no lock at all

atomic.c – for a single counter there is a third solution, which uses no lock at all. The type atomic_long, declared in stdatomic.h, which comes with gcc and needs nothing to be installed, is an integer on which every operation is indivisible, and atomic_fetch_add performs the load, the addition and the store as one single step of the processor, so that no thread can cut in between them. The race of the first part therefore cannot happen, and no thread ever waits for another: the hardware does the work that the lock was doing. This solution is the fastest of the correct ones, but it only works because the critical section is a single arithmetic operation on a single variable; as soon as two variables must be changed together, a lock is needed again. The source is given below.

```c
/* atomic.c - the counter of race.c, with no lock at all: one atomic instruction
 * per addition.
 * Build: gcc -Wall -pthread -o atomic atomic.c
 */
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>

#define N 1000000

atomic_long counter = 0;       /* every operation on it is indivisible */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        atomic_fetch_add(&counter, 1);   /* load, add, store as ONE step */
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", (long)counter, 2 * N);
    return 0;
}
```

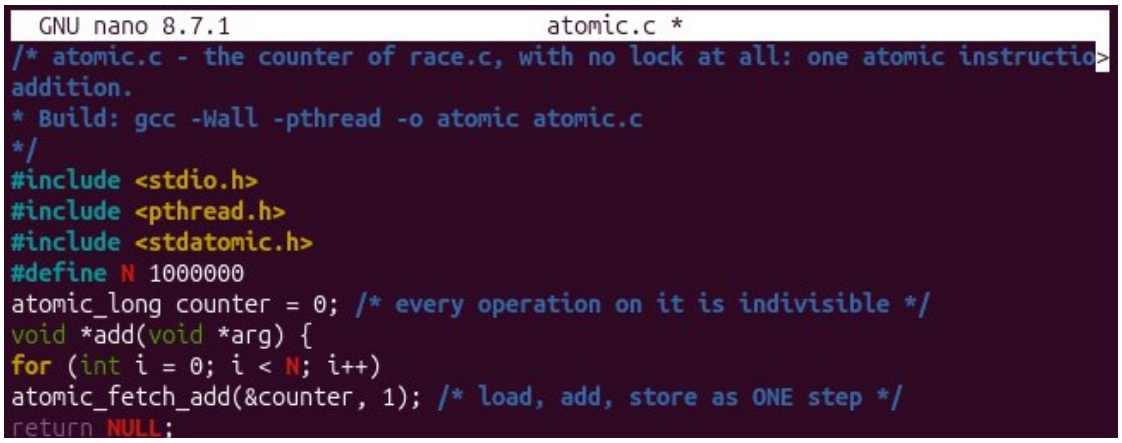


Figure 5.1: The beginning of atomic.c in the editor

nano atomic.c – the program is the one of the first part with two changes: the type of the variable and the way it is incremented. There is no mutex, no semaphore and no header of the threads beyond the one already needed to create them, which makes the comparison with race.c direct: the same two threads, the same million additions each, and a correct result. This part is marked as optional in the task sheet and is not graded, and the time of the run was not measured here [TODO].

Conclusion

The laboratory work showed that a line as short as counter++ is not one action but three, and that this is enough to lose work when two threads execute it at the same time. In the first part the unprotected program printed 1000000, 1559265 and 1293920 instead of 2000000, three wrong and three different numbers, because the three steps of one addition were interleaved with those of another and a store overwrote a value that had already been read; the result changes at every run because the interleaving depends on the scheduler and not on the program.

In the second part the same counter was repaired twice. The mutex, placed around the single line of the critical section, and the semaphore started at 1 both printed exactly 2000000, and the correctness is now guaranteed rather than probable, because the interleaving that caused the loss has become impossible. A semaphore started at 2 would destroy that guarantee, since it would let both threads in at once, which is why the start value of a lock must be 1. The measurements with time showed what the guarantee costs: 0.013 s for the wrong program, 0.104 s for the mutex and 0.259 s for the semaphore, the difference between the last two lying almost entirely in the system time, 0.045 s against 0.264 s, because a mutex settles most of its work in the user space while the semaphore calls the kernel more often.

In the third part the problem was the order and not the simultaneity, and a semaphore started at 0 solved it: the reader blocked at sem_wait, the one second of the writer passed visibly, and only after sem_post did the reader print the message it had been waiting for. Started at 1 instead, the same program would let the reader through immediately and it would print an empty message, in the wrong order and without any error. In the fourth part the semaphore started at 3 counted the free places of a parking lot with six cars, and the printed value of inside rose to 3, fell and rose again, but never to 4; with one place it would simply become a mutex. The three parts together showed that the semaphore is a single mechanism, a number that never falls below zero, which becomes a signal, a lock or a counter depending only on the value it starts from; all the objectives of the laboratory work were achieved.

References

1. *Gabriela Pricop. os-2026-GabrielaPricop: laboratory works of the Operating Systems course, GitHub repository*
   `https://github.com/gabriela-pricop/os-2026-GabrielaPricop`
2. *pthreads(7) – POSIX threads, an overview of the thread library, Linux manual page*
   `https://man7.org/linux/man-pages/man7/pthreads.7.html`
3. *sem_overview(7) – overview of POSIX semaphores, Linux manual page*
   `https://man7.org/linux/man-pages/man7/sem_overview.7.html`
4. *pthread_mutex_lock(3) – lock and unlock a mutex, Linux manual page*
   `https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3.html`